

1 The contract, adopted and not forked

The plugin's full tree: one display/ per page, one folder per unit, every unit the same shape.

```
<page>/
  <page>.md          the page, on stage
  display/          the plugin . discovery never enters
    <PageID>-Display1-<slug>/    ONE rendered unit
      README.md      claim . kind . caption-job . fragility . status
      intake/        caller-owned, provenance-bound values
        manifest.yaml
        inputs/      small approved summary CSV or JSON extracts only
      recipe/        renderer-owned script, spec, receipts
      float.tex      caption + label + asset reference
      preview.tex    standalone wrapper that inputs float.tex
      preview.pdf    compiled look, what the tab frames
      assets/        the WINNING render
      candidates/    pre-decision renders
      versions/      superseded variants, kept for history
    <PageID>-Display2-<slug>/    the next unit, same shape
```

The layout above is `display-unit-output-contract.md`, quoted rather than redesigned. `intake/` and `recipe/` are primary material a person rules; `assets/` and `preview.pdf` are derived and a rebuild may overwrite them. The one page-side change is the address: the unit root is the page's own `display/` folder, so the unit travels with the page it illustrates.

2 The surface: one card per unit in the right pane

The tab: what a reader sees when they open Display on a page.

```
right pane . Display
+-----+
| QPf5-Display1-drift-bands          |
|   preview.pdf, framed (body.py ccpdf) |
|   README status . rebuild          |
|   accepted: [ ] . shown, NOT tickable |
+-----+
one card per unit . empty display/ = empty state, not an error
```

The tab is the Slides sandwich with a different filling: a drawer plugin posts to a `live/` endpoint, the endpoint lists the page's units, and the pane frames each `preview.pdf` through the PDF object path `src/body.py` already renders. `rebuild` runs the unit's recipe and refreshes the derived half; it never touches `intake/` and never touches an acceptance row. A unit is EVIDENCE the content cites, and the citation lives IN the sentence: the unit's id named in prose chips as the evidence card in place, carrying its acceptance state and linking its block in the strip. The `> Display:` lane beside `> Citation:` and `> Value:` stays the FILING surface: a machine appending evidence writes a lane and never edits prose, and a binding no sentence carries naturally lands there. Either surface naming a `[ ]` unit binds a pending render, not accepted evidence. The projections inherit the

citation (JL 260816): the latex export embeds a cited unit as a real float after the citing paragraph, built from the winning asset and the unit's own caption, and the word export embeds the rasterized figure with an inline figure number and a `Display` comment on the sentence.

3 The writers, and the row no machine may tick

Who writes what: three hands on one unit, each confined to its half.

```

renderer skills    haipipe-display-table . -figure . -diagram . ...
                  write recipe/ . assets/ . preview.pdf
a person          rules intake/ and ticks accepted:
the pane          shows everything . writes NOTHING

```

The renderers are the display family's existing skills, dispatched the way `task-for-display` already dispatches them; this page adds no renderer. Acceptance follows the model the slide plugin already borrowed: a `[x]` means a person looked at a specific render and said yes, so a machine that ticks one has forged a judgment, not saved time.

The four renderers: which skill draws which kind, and what each one's recipe is.

```

data-driven . needs a ruled intake/ of numbers
  -table      aggregated CSV/JSON > booktabs LaTeX . recipe = the spec
  -figure     results > plot . recipe = the python script
concept-driven . the intake is a spec, not numbers
  -diagram    FigureSpec JSON > deterministic editable SVG
  tikz        .tikz.tex recipe > vector PDF . TeX-native: the paper's
              fonts, math labels, and float.tex may input it directly
  -illustration image-gen prompt > concept figure . recipe = prompt + receipts
  -poster . -slides composites, assembling units the renderers above made

```

A data-driven renderer may not recompute from raw data; it reads the approved extract in `intake/` and nothing else. A concept-driven renderer carries no numbers, so its intake is the spec it draws. The rebuild differs the same way: a figure rebuild reruns its python script, a diagram rebuild is deterministic from its spec, and an illustration rebuild replays a receipted prompt.

4 How a unit is generated

The pipeline: five steps from approved numbers to an accepted render, and who moves each one.

```

(1) INTAKE      the caller fills intake/: manifest.yaml + small
                 approved extracts, every value provenance-bound
(2) RENDER      the matching renderer skill runs, dispatched through
                 task-for-display; it fills recipe/ + candidates/
(3) PICK        a person picks the winner; it moves to assets/,
                 the losers stay in candidates/ or versions/
(4) BUILD        preview.tex compiles to preview.pdf; the adapter's
                 --check reports a stale unit instead of hiding it
(5) ACCEPT      a person reads the render and ticks accepted:

```

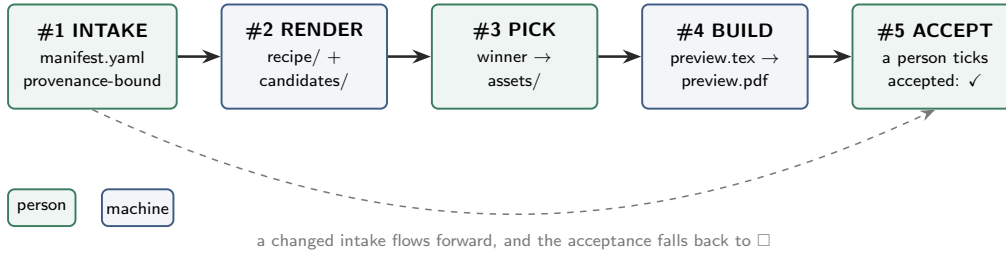


Figure 1: The display pipeline: five steps from approved intake to an accepted render. Green steps belong to a person, blue steps to machinery, and a changed intake returns the acceptance to open.

The pipeline never starts at (2): a renderer given no ruled `intake/` has nothing provenance-bound to draw, which is what `display-intake-contract.md` exists to refuse. Steps (1) (3) (5) are a person’s and steps (2) (4) are machinery, the split of hands QPf5-Display1 draws; rebuild on the tab is step (4) alone, rerun. A changed `intake/` flows forward, (2) to (5), and the acceptance falls back to `[]` because the bytes a person said yes to are gone.

Where (1)’s numbers come from: the evidence chain behind an intake, id-bound at every hop.

the page needs a number	> raised as a PROBE . the page's own probe/ plugin (QPf9)
probe orchestrator	> sweeps the bank . reuse enrich fresh
haipipe-task runs	> answer lands in <task-folder>/QA/<n>-<slug>.md
	+ the run's artifact, e.g. counts.csv . CANONICAL
intake/manifest.yaml	> cites that holder BY ID . carries only a
	small approved extract in inputs/

`intake/` is not a second data store: the task output stays canonical, and the manifest points back at the exact holder and run that produced it. A unit names its evidence by id, never by path, and never types a number; QPf5-Display2 draws this ask-once-cite-twice move. The slide plugin already walks this chain on QPf3: its deck declares `needs: QA-probe/QBt5-for-value/1-artifact-paths`, the resolver turns the id into the artifact’s path, and the template carries no digits, so a hand-typed value has nowhere to live. A1’s adapter is what makes the same id resolution work for a unit under a page.

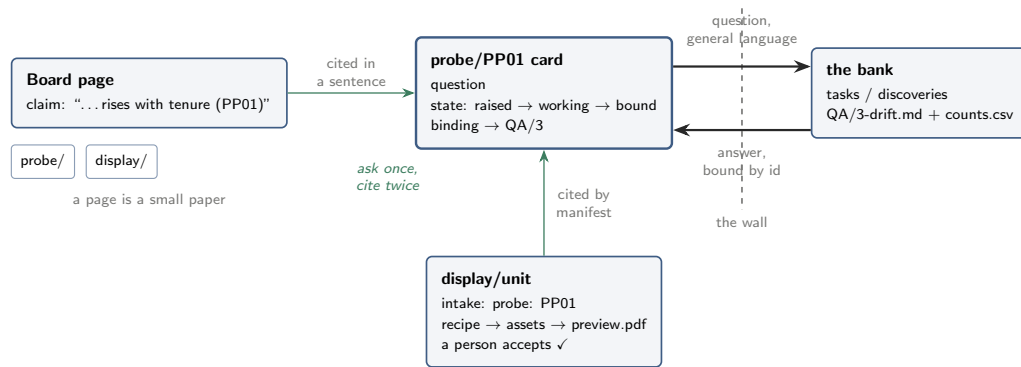


Figure 2: A page as a small paper: the probe plugin asks a question once, the bank answers behind the wall, and both the display intake and the page's own sentence cite the same card by id.